

Optimising performance on NVIDIA CUDA

Get more from your GPUs with a few simple tips

Mithun Mohandas | mithun@digit.in

If you're interested in machine learning or AI, then there's nothing more popular than NVIDIA's CUDA architecture. Globally, NVIDIA GPUs power most of the AI processing hardware and as of Q3-2021, they enjoyed a market share of 80.6 per cent of the global revenue, as per OMDIA. That's 3.2 billion dollars! Xilinx, Google, Intel and AMD follow NVIDIA, although the market has changed quite a bit since AMD acquired Xilinx. Nevertheless, you will have to learn how to program for CUDA cores at some time or the other. We're hoping to help you optimise your code for performance with these few simple guidelines.

One of the principal methodologies for determining which strategy you need to follow relies on careful examination of the code and its performance. Different portions of the code base will react differently with the hardware. The goal is to identify portions of the applications that need optimisation and then run performance profiling tools to see if there are any performance limiters for said portion. For example, a portion of the code may be limited by the available memory primarily. Optimising this portion for parallelisation or instruction throughput is not going to yield much of a performance improvement. You will have to switch to a better hardware with more memory to overcome a memory limit.

Going along these lines, there are several basic strategies for optimising performance. These are:

- Maximise utilisation through parallelisation
 - Maximise memory throughput
 - Maximise instruction throughput
 - Minimise memory thrashing
- Let's go through these strategies to

see how they can be implemented to give you that little extra boost of performance.

MAXIMISE UTILISATION

One of the key tenets of GPGPU is parallelisation. Modern graphics cards sport hundreds or even thousands of similar computing cores which can all run in parallel to maximise processing throughput. By leveraging the parallelisation capabilities and maximising utilisation across all available parallel paths, you can improve performance by leaps and bounds. The goal is to keep as many CUDA cores busy for most of the time.

CUDA allows for the following operations to be treated as independent tasks that can run concurrently:

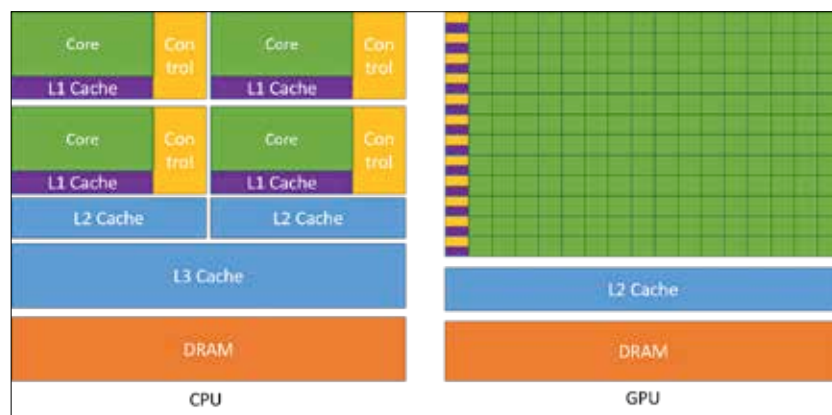
- Computation on the host
- Computation of the devices
- Memory transfer from host to device
- Memory transfer from device to host
- Memory transfer within memory of device

Memory transfer across devices

In this context, the host is the computer in which all the devices a.k.a. the graphics cards are installed. And modern day GPUs including the ones from NVIDIA allow for multiple graphics cards to be installed in the same host. Parallelised workloads can be shared across all the compatible graphics cards installed in a host machine.

Application Level

An application being written for CUDA should maximise parallel execution of the code between all the elements mentioned above. This includes the host, the devices and the bus interface that connects all the devices to the host. This can be done via Asynchronous Concurrent Execution. The host is usually best equipped to handle serialised workloads whereas the devices i.e. the



CPU vs GPU: Potential for parallelisation